

Name: Charmi Gaddale

Roll Number: 23071A0516

AIM: Implement k-Nearest Neighbor (k-NN).

Dataset: Breast Cancer

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
from sklearn.datasets import load_breast_cancer

# Split dataset
from sklearn.model_selection import train_test_split

# Feature scaling
from sklearn.preprocessing import StandardScaler

# k-NN model
from sklearn.neighbors import KNeighborsClassifier

# Evaluation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
```

```
# Load Breast Cancer dataset
data = load_breast_cancer()

# Features (medical measurements)
X = data.data

# Target labels (0 = malignant, 1 = benign)
y = data.target

# Convert dataset into DataFrame
df = pd.DataFrame(X, columns=data.feature_names)

# Add target column
df["target"] = y

# Show first few records
print(df.head())

# Show total records
print("Total records:", df.shape[0])
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	

2	0.05999	...	25.53	152.50	1709.0
3	0.09744	...	26.50	98.87	567.7
4	0.05883	...	16.67	152.20	1575.0

	worst smoothness	worst compactness	worst concavity	worst concave points	\
0	0.1622	0.6656	0.7119		0.2654
1	0.1238	0.1866	0.2416		0.1860
2	0.1444	0.4245	0.4504		0.2430
3	0.2098	0.8663	0.6869		0.2575
4	0.1374	0.2050	0.4000		0.1625

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]
Total records: 569

```
# Split dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Standardize the dataset
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Create k-NN model with k = 5 neighbors
```

```
knn = KNeighborsClassifier(n_neighbors=9)
```

```
# Train the model
knn.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=9)

```
# Predict using test dataset
```

```
y_pred = knn.predict(X_test)
```

```
# Model accuracy
```

```
print("k-NN Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Detailed classification metrics
```

```
print(classification_report(y_test, y_pred))
```

```
k-NN Accuracy: 0.9649122807017544
```

	precision	recall	f1-score	support
0	0.95	0.95	0.95	43
1	0.97	0.97	0.97	71
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114

```
# Generate confusion matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```

# Plot confusion matrix
plt.figure(figsize=(6,4))

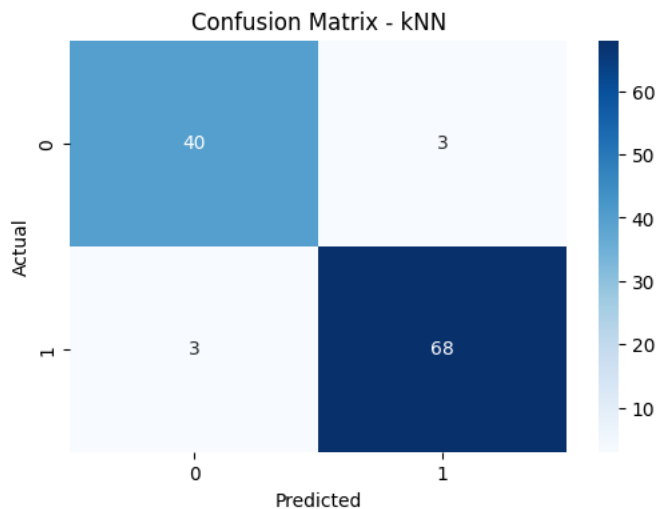
sns.heatmap(cm, annot=True, cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.title("Confusion Matrix - kNN")

plt.show()

```



```

accuracy_scores = []

# Test K values from 1 to 15
for k in range(1,16):

    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    pred = model.predict(X_test)

    accuracy_scores.append(accuracy_score(y_test, pred))

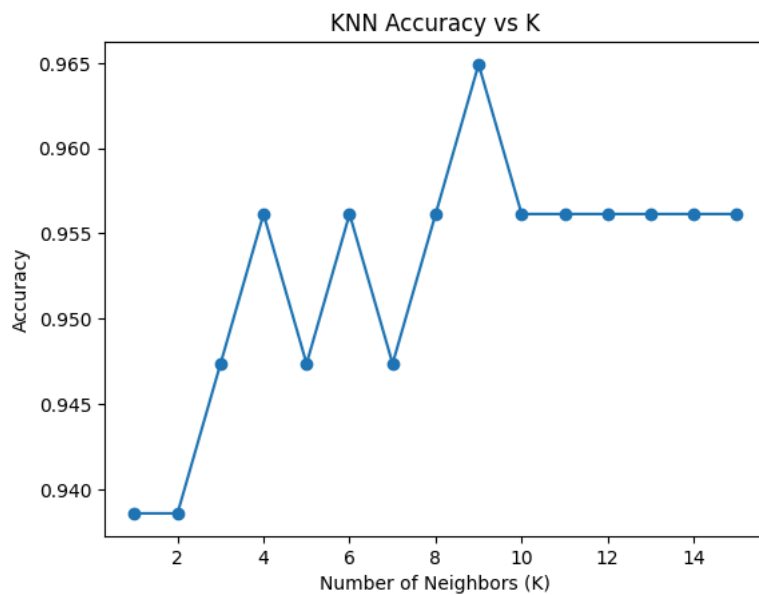
# Plot K vs Accuracy
plt.plot(range(1,16), accuracy_scores, marker='o')

plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")

plt.title("KNN Accuracy vs K")

plt.show()

```



```
# Example prediction using a new sample

sample = [X_test[0]]

prediction = knn.predict(sample)

if prediction[0] == 0:
    print("Prediction: Malignant Tumor")
else:
    print("Prediction: Benign Tumor")
```

Prediction: Benign Tumor

Inference: In this experiment, the k-Nearest Neighbors algorithm was implemented using the Breast Cancer dataset. The dataset was split into training and testing sets and standardized before training the model. The k-NN algorithm classifies data points based on the majority class among the nearest neighbors. The model performance was evaluated using accuracy, classification metrics, and a confusion matrix. The results show that the k-NN algorithm can effectively classify tumors as malignant or benign based on the nearest neighbor principle.